微步在线

1. OSI七层模型（从上到下）

7. **应用层**：给应用程序提供网络服务，HTTP、HTTPS、DNS、FTP
8. **表示层**：数据编码、加密解密、序列化、格式转换（如JSON、SSL/TLS早期放在这一层理解）
9. **会话层**：建立、管理、断开会话连接
10. **传输层**：端到端传输，TCP、UDP，端口号
11. **网络层**：寻址、路由，IP、ICMP，路由器
12. **数据链路层**：MAC帧、差错检测，交换机
13. **物理层**：比特流、网线、信号电平、光纤

> 补充面试话术：实际互联网使用TCP/IP四层模型：应用层、传输层、网际层、网络接口层。

2. HTTP请求方法

- **GET**：查询资源，参数放在url，幂等，不应该修改服务端数据
- **POST**：提交数据、创建资源，非幂等，请求体传参
- **PUT**：全量更新资源，幂等
- **PATCH**：局部更新资源（部分字段修改）
- **DELETE**：删除资源，幂等
- **HEAD**：和GET类似，只返回响应头，不带响应体，用来探测资源
- **OPTIONS**：查询服务器支持哪些请求方法（跨域预检请求用它）
- **TRACE**：回环测试，追踪报文链路，一般服务器默认禁用，有安全风险
- **CONNECT**：建立隧道，HTTPS代理使用

> 重点区分幂等：同一个请求执行多次，服务端结果和执行一次一样。
> GET/PUT/DELETE/HEAD/OPTIONS 幂等；POST默认不幂等。

3. 0day漏洞（零日漏洞）

口述简洁版：
0day漏洞，就是厂商还不知道、没有补丁的安全漏洞。

- 含义：漏洞被发现时，软件厂商**零天（0 day）修复时间**，没有官方补丁，用户无法通过升级、打补丁防护；
- 来源：安全研究员、黑客挖掘得到；
- 风险很高：攻击者可以利用它发起攻击，而厂商和用户毫无防备；
- 对比：
  - 0day：无补丁；
  - nday（1day/7day）：漏洞已经公开、厂商已经发布补丁，只是很多用户没来得及修补；
- 延伸（微步是安全厂商，加分）：微步的威胁情报里，会持续捕获0day利用样本、IOC，做检测和告警；0day一般会被用于高价值定向攻击（政企、能源等）。

1. 快速排序、冒泡排序

快速排序

思想：**分治 + 分区**

1. 选一个基准 pivot；

2. partition：把小于pivot放左边，大于pivot放右边；
3. 递归对左右两个子数组重复分区。

- 时间：平均 O(nlogn)，最坏 O(n^2)（有序数组）
- 不稳定排序；不需要额外辅助空间（原地快排）

冒泡排序

思想：相邻元素两两比较，逆序就交换，每一轮把最大元素“冒泡”到末尾。

- 时间：最好O(n)（优化版带flag），最坏O(n^2)
- 稳定排序；适合小规模有序数据，工程几乎不用

2. MySQL主键和外键区别

- **主键 PRIMARY KEY**：唯一标识一行记录，**非空、唯一**；一张表只能有一个主键；InnoDB聚簇索引依托主键；用来唯一定位本行数据。
- **外键 FOREIGN KEY**：建立两张表的关联，引用另一张表的主键；用来保证参照完整性；可以多个，可以为null；**外键约束保证子表引用的数据在主表一定存在。**

> 补充：很多业务开发会逻辑上做关联，业务层校验，不在数据库开启外键（外键影响插入更新性能、容易锁表）

3. TCP三次握手、四次挥手

✅ 三次握手（建立连接）

1. 客户端 → 服务端：SYN（seq=x），请求建立连接
2. 服务端 → 客户端：SYN+ACK（seq=y，ack=x+1）
3. 客户端 → 服务端：ACK（ack=y+1），连接就绪

✅ 四次挥手（断开连接，TCP全双工）

1. 客户端发 FIN：客户端不再发数据（还能收）
2. 服务端回 ACK
3. 服务端数据发完，发 FIN
4. 客户端回 ACK，进入 TIME_WAIT（2MSL）

为什么四次：TCP双向独立关闭，服务端收到FIN后可能还有剩余数据要发给客户端，不能和ACK合并。

4. TCP 和 UDP 的区别

1. TCP面向连接；UDP无连接
2. TCP可靠传输；UDP不可靠，不保证到达、不乱序
3. TCP有流量控制、拥塞控制；UDP没有
4. TCP是字节流；UDP是数据报
5. TCP头部大，开销高；UDP头部极简，速度快

TCP可靠手段：序列号+ACK确认、超时重传、滑动窗口、校验和、拥塞控制。
UDP适用：DNS、音视频直播、QUIC。

5. HTTP状态码（高频必记）

- 1xx 信息：100 Continue
- 2xx 成功
  - 200 OK：成功
  - 201 Created：创建成功（POST）

    - 204 No Content：成功无返回体
- 3xx 重定向
    - 301：永久重定向
    - 302：临时重定向
    - 304 Not Modified：缓存命中，资源未修改
- 4xx 客户端错误
    - 400 Bad Request：请求参数错误
    - 401 Unauthorized：未认证
    - 403 Forbidden：认证了，但是没权限
    - 404 Not Found：资源不存在
    - 405 Method Not Allowed：请求方法不支持
    - 413、429 Too Many Requests：限流
- 5xx 服务端错误
    - 500 Internal Server Error：服务内部异常
    - 502 Bad Gateway：网关拿到无效响应
    - 503 Service Unavailable：服务不可用、过载、下线
    - 504 Gateway Timeout：网关超时

如果你需要，我把这一批压缩成一句话极简速记版。

微步在线Java1

> 说明：回答偏向后端开发视角，不往安全研究员方向过度延伸，重点突出Java后端、线上排查、性能调优，匹配JD高并发、性能优化的要求

一、操作系统

1. 进程和线程的核心区别是什么?

进程是**资源分配的最小单位**;

线程是**CPU调度的最小单位。**

进程之间相互隔离；线程隶属于同一个进程，共享进程资源，切换代价更小。

一句话总结：进程是独立的程序运行实例，线程是进程内的执行流。

2. 它们共享和独享哪些资源?

✅ 同进程内线程**共享**： 地址空间（堆、全局变量、方法区）、文件描述符、信号处理、进程ID、内存映射。

✅ 线程**独享**： 栈空间、程序计数器PC、寄存器上下文、线程本地存储TLS（ThreadLocal）。

✅ 进程之间：资源基本隔离，需要IPC（管道、socket、共享内存等）通信。

3. 一个进程里有多个线程，一个线程崩溃了，为什么有可能带崩整个进程?

线程运行在同一个进程地址空间：

1. 如果线程触发**段错误、访问非法内存**，操作系统给整个进程发SIGSEGV信号，进程直接退出；
2. 线程异常没有捕获，JVM中如果非守护线程抛出未捕获异常，线程停止；**如果是OOM这类共享内存资源耗尽，整个进程所有线程都无法分配内存，全部挂掉**；

3. 多个线程共用锁、共享资源，线程崩溃时没释放锁，其他线程死锁卡死，服务不可用。

> 补充（Java场景）：普通业务线程抛Exception如果有try-catch不会崩；但Native代码、JVM内部异常、OOM会让整个JVM宕机。

4. 大概有哪些类型？（进程/线程状态）

进程/线程通用五态：新建、就绪、运行、阻塞（等待）、终止。

- 就绪：等待CPU时间片；
- 运行：正在CPU上执行；
- 阻塞：等待IO、锁、信号，不抢占CPU。

5. 什么情况下判断多线程性能好？什么时候线程很多性能反而差，怎么判断？

✅ 性能好：CPU利用率合理、吞吐量上升、延迟稳定，没有大量上下文切换、没有频繁锁竞争。线程数匹配负载（IO密集型可以多一点，CPU密集型≈CPU核数）。

❌ 线程很多性能差的典型： 大量**上下文切换、自旋锁空转、锁争抢激烈**，大量时间浪费在线程调度而不是业务计算。

👉 判断手段： Linux：`top`、`pidstat -w`看ctxt切换次数；`perf`采样；Java侧：Arthas看线程状态，大量WAITING、BLOCKED线程，CPU us不高但sy（内核CPU）很高，大概率线程过多。

6. 线程上下文切换带来哪些成本？怎么判断切换多？

成本：

1. 保存/恢复寄存器、PC、栈上下文；
2. TLB、CPU缓存失效（缓存命中率下降）；
3. 内核态用户态切换开销。 判断：`pidstat -w`看`cswch/s`（自愿切换，阻塞导致）、`nvcswch/s`（非自愿切换，时间片耗尽、被抢占）；perf看sched_switch事件。sy内核CPU占比很高也是信号。

7. 单核CPU能不能并发运行多个线程？

可以并发，不能并行。 单核CPU靠时间片轮转调度，快速切换线程，宏观上看起来同时跑，微观同一时刻只有一个线程在执行。

> 并发：交替执行；并行：同一时刻多核同时执行。

8. 用户态线程/协程为什么减少切换成本？了解协程吗？

内核线程切换需要陷入内核、操作系统调度； 协程（用户态轻量级线程）**调度在用户空间完成，不需要内核参与**，不需要触发系统调用、不用更换进程页表，上下文极小，切换成本远低于内核线程。 Java：虚拟线程（JDK21+）就是协程思想；Go goroutine也是典型协程。 局限：单核下协程如果有阻塞的系统调用（如阻塞IO），依然会绑定承载的内核线程阻塞。

9. CPU亲和性（CPU绑定）了解吗？

CPU亲和性：把一个线程绑定到指定的一个或者几个CPU核上运行。 好处：减少线程在不同核之间漂移，减少TLB、Cache失效，降低上下文切换带来的缓存损失； 风险：绑定不合理会造成核负载不均衡。Java、Nginx、中间件常用这个优化。

二、计算机网络

12. IO多路复用哪几个函数？

Linux下：`select`、`poll`、`epoll`（最常用）； macOS/BSD：`kqueue`；Windows：IOCP。 重点讲epoll：水平触发LT、边缘触发ET。

> 一句话：单个线程通过这一组函数同时监听多个文件描述符的IO就绪事件。

13. 网络七层模型

从上层到下层： 7 应用层（HTTP、DNS） 6 表示层（编码、加密、序列化） 5 会话层（会话建立维护） 4 传输层（TCP/UDP） 3 网络层（IP、ICMP） 2 数据链路层（MAC、以太网帧） 1 物理层（电平、网线、比特流）

> 面试也可以顺带提：TCP/IP四层（网络接口层、网际层、传输层、应用层）

14. 交换机属于哪一层?

二层，**数据链路层**，基于MAC地址转发帧。

15. 路由器哪一层？转发原理?

三层，**网络层**，基于IP地址转发数据包。 原理：维护路由表，收到数据包，取出目标IP，查路由表匹配下一跳，修改二层MAC头，转发到对应出口；跨网段隔离广播域。 作用：连接不同网段，实现跨局域网通信。

16. 四层负载均衡、七层负载均衡，在哪一层，区别?

- 四层：传输层（TCP/UDP），代表LVS。只看IP+端口，不解析应用层协议，转发快，无法感知HTTP路径、Cookie；直接转发报文。
- 七层：应用层，代表Nginx。解析HTTP协议，可以根据url、header、cookie做转发策略；代价更高，性能低于四层LB。

17. TCP三次握手、四次挥手

三次握手（建立连接）

1. 客户端发SYN，seq=x；
2. 服务端回复SYN+ACK，seq=y，ack=x+1；
3. 客户端回复ACK，ack=y+1。

四次挥手（断开连接）

1. 客户端FIN，关闭写方向；
2. 服务端ACK；
3. 服务端处理完剩余数据后发FIN；
4. 客户端ACK，进入TIME_WAIT。

18. 为什么关闭连接需要四次挥手?

TCP是**全双工通道**，两个方向独立关闭： 一方发FIN，代表它不再发送数据，但还可以接收对方剩下的数据。 服务端收到FIN之后，先回ACK，此时服务端还能继续给客户端发剩下业务数据；等服务端数据发完，自己再发FIN。所以不能合并成三次。

19. TIME_WAIT的作用

客户端最后ACK之后等待2MSL才关闭：

1. 保证最后一个ACK报文能到达服务端；如果ACK丢了，服务端超时重发FIN，客户端还能再次回复ACK；
2. 等待网络中该连接的残留旧报文消失，避免新复用端口的连接收到旧连接的脏数据包。

> 痛点：高并发短连接场景大量TIME_WAIT占满端口，是后端线上常见问题。

三、Java开发中的高可用和高并发

> 面试口述版 **高并发**：系统能够同时承载大量请求，重点是吞吐、延迟、QPS。 手段：池化（线程池、连接池）、异步、缓存（Redis）、分库分表、索引优化、消息队列削峰、读写分离、限流、降级。 **高可用**：尽量保证服务不中断，故障之后可以快速恢复，核心目标是减少宕机时长。 手段：集群多实例、无状态设计、健康检查、熔断、故障自动转移、主从切换、异地多活、限流保护、优雅关闭、日志监控告警。

一句话区分：高并发扛流量；高可用扛故障。

四、安全与高可用（微步是安全公司，这块重点准备）

22. XSS 和 CSRF 攻击原理

✅ XSS（跨站脚本）： 攻击者往页面注入JS脚本，浏览器渲染页面时执行恶意JS；窃取Cookie、劫持会话、钓鱼。 分类：存储型、反射型、DOM型。 防御：输入过滤、输出转义、CSP策略、HttpOnly Cookie。

✅ CSRF（跨站请求伪造）： 利用浏览器会自动带上目标站点Cookie的特性，诱导用户在已登录状态，访问攻击者页面，悄悄发起请求（比如转账、改密码）。 防御：Token校验（CSRF Token）、校验Referer、SameSite Cookie。

> 重点区分：XSS是**执行恶意代码**；CSRF是**冒用用户身份发起请求**。

23. TCP半连接队列、全连接队列

三次握手过程：

1. 客户端发SYN → 服务端放入**半连接队列（syn queue）**，状态SYN_RCVD；
2. 服务端收到ACK，握手完成，从半连接队列移出，加入**全连接队列（accept queue）**；
3. 应用程序调用accept()从全连接队列取出连接。

- 半连接队列：存放未完成三次握手的连接；SYN洪水攻击就是打满这个队列。
- 全连接队列：握手完成、等待应用层accept的连接；队列满了，新连接会丢包，线上会出现建连失败。 Java NIO、Netty、Nginx都有这两个队列参数可以调。

24. 高并发开发哪一部分比较重要？挑着讲

（推荐优先讲这套，贴合你简历和JD） 我认为**缓存设计 + 锁与并发控制 + 数据库性能**三块是核心：

1. 缓存：Redis，解决DB压力；要考虑缓存穿透、击穿、雪崩、一致性问题；
2. 并发控制：JUC锁、无锁、线程池合理配置，避免大量竞争、死锁、上下文切换；区分IO密集/CPU密集线程模型；
3. 数据库：索引优化、慢SQL、分页、事务隔离级别，防止行锁、表锁拖垮整个服务； 其次还有限流降级、异步队列削峰、合理的负载均衡。

> 备选：也可以说IO模型（NIO、epoll），刚好和前面网络、IO多路复用串起来。

25. 你怎么去保证skill的输出是一致的?

> 这题偏向AI/大模型相关（微步也在做安全大模型XGPT），后端视角回答： 如果这里的skill指大模型工具/函数能力：

1. 首先给skill明确、固定的Prompt、输入输出Schema，强结构化输出（JSON），限制自由文本；

2. 增加结果校验层：输出之后做规则校验、格式校验、值域校验，不符合就重试；
3. 增加RAG知识库兜底，给固定标准参考资料，减少幻觉；
4. 相同输入增加重复采样，做结果一致性投票；
5. 版本管理：skill能力、prompt、工具逻辑版本化，不随意变更逻辑；
6. 线上灰度、监控：统计输出异常率，异常自动告警。

> 简化简短版： 通过固定输入输出schema、结构化返回、结果校验、参考资料约束、版本管控、采样投票的方式，减少随机性，保证输出稳定一致。

微步在线2

> 适配校招后端面试，兼顾原理+线上场景，简洁好背

1. StringBuilder 和 StringBuffer 的区别

- **StringBuffer**：方法加了 `synchronized`，**线程安全**；同步锁带来开销，性能偏弱
- StringBuilder：无同步锁，线程不安全；性能更高，单线程场景优先使用

  共同点：底层都是可变char数组（JDK9+改为byte数组），用于字符串拼接，避免频繁创建String对象。

  > 一句话总结：安全与否是核心差异，单线程用StringBuilder，多线程共享才考虑StringBuffer。
  >
  > 大多数实际开发中更常用 `StringBuilder`，因为字符串拼接通常发生在方法内部，是局部变量，不会被多个线程共享。

2. 遇到过哪些线程安全问题?

结合你的实习场景口述，举典型例子：

1. **ArrayList 多线程add**：扩容竞争、数组越界、数据丢失（ArrayList非线程安全）
2. **SimpleDateFormat**：共享实例并发调用，时间解析错乱，是高频踩坑点
3. 静态共享变量无锁，多个线程同时修改，数据覆盖
4. 线程池里使用非线程安全集合，共享缓存变量 配套解决方案：

- 用UC包：`CopyOnWriteArrayList`、`ConcurrentHashMap`
- 局部变量（栈内变量天然线程安全，优先）
- 加锁synchronized / ReentrantLock；ThreadLocal隔离变量

3. ArrayList 和 LinkedList 的区别？LinkedList是双向还是单向链表?

区别

1. 底层：ArrayList **动态数组**；LinkedList **双向链表**
2. 随机访问：ArrayList支持下标O(1)随机读；LinkedList必须遍历O(n)
3. 插入删除：尾部插入ArrayList很快；**中间位置插入删除LinkedList更快（不需要数组拷贝）**；头尾增删LinkedList优势明显
4. 内存：ArrayList连续内存，CPU缓存友好；LinkedList节点分散，每个节点多存前驱后继指针，内存开销更大
5. 扩容：ArrayList满了会扩容、拷贝数组；链表没有扩容概念

链表结构

LinkedList 是**双向链表（Doubly Linked List）**，每个节点包含：`item数据`、`prev前驱指针`、`next后继指针`。

> 链表基础：单向链表只有next；双向链表同时有prev、next，可以向前遍历。

4. HashMap 原理？hash定位数组？必须重写hashCode和equals吗？

底层（JDK8）

数组 + 链表 + 红黑树。链表长度≥8，数组长度≥64时链表转红黑树；树节点≤6退化为链表。

定位桶下标

```
hash = key.hashCode() ^ (hash >>> 16);  // 扰动函数，高低位混合，减少hash碰撞
index = hash & (table.length - 1);      // 代替取模%，table容量是2的n次幂
```

hashCode & equals

必须成对重写：

- 如果只重写equals不重写hashCode：两个逻辑相等的对象，hashCode大概率不同，会分到不同桶，HashMap认为是两个不同key，造成重复存储。
- 如果只重写hashCode不重写equals：hash相同的不同对象，equals返回false，会判定为不同key。

> 规则：equals相等的对象，hashCode一定相等；hashCode相等，equals不一定相等（hash碰撞）。

5. 单例模式怎么实现？挑常用两种讲

① 双重检查锁 DCL（推荐，工程常用）

```
public class Singleton {
    private static volatile Singleton instance;
    private Singleton(){}
    public static Singleton getInstance(){
        if(instance == null){
            synchronized (Singleton.class){
                if(instance == null){
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

volatile作用：禁止指令重排，防止半初始化对象逸出。

② 静态内部类（懒加载、无锁，简洁推荐面试说）

```
public class Singleton {
    private Singleton(){}
    private static class Holder{
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance(){
        return Holder.INSTANCE;
    }
}
```

利用类加载机制，内部类只有调用时才加载，天然懒加载、线程安全。

> 不推荐饿汉：提前加载占用资源；不推荐枚举（可以提一句：枚举单例天然防反射、序列化破坏）

6. Java类执行顺序（两层：父类、子类）

优先级：**静态 > 非静态（实例） > 构造方法** 执行顺序：

1. 父类静态代码块、父类静态变量（只加载一次）
2. 子类静态代码块、子类静态变量
3. 父类实例变量、父类普通代码块
4. 父类构造函数
5. 子类实例变量、子类普通代码块
6. 子类构造函数

> 静态属于类，类加载阶段执行；实例代码块、构造器是new对象的时候执行。

7. 用过哪些Java命令？遇到过什么实际问题？

常用命令：

- `java`：运行程序
- `javac`：编译
- `jps`：查看java进程pid
- `jstack`：打印线程栈 → 排查死锁、线程卡死、BLOCKED、WAITING
- `jmap`：dump堆快照，看对象内存占用
- `jhat`/MAT：分析dump文件
- `jstat`：监控GC、内存变化
- `jinfo`：查看JVM参数
- `arthas`（工具，顺带说，加分）

实际问题举例： 线上CPU高，用`jstack`定位到死循环；OOM时`jmap -dump`导出堆文件，MAT分析大对象、内存泄漏；线程死锁jstack直接抓到死锁信息。

8. Linux查看Java进程占用端口

```
# 方式1
netstat -tlnp | grep java
# 方式2（推荐，新版）
ss -tlnp | grep java
# 根据pid查端口
lsof -p <pid>
```

9. Linux查看Java进程打开了哪些文件

```
lsof -p <pid>
```

可以看到打开的日志文件、socket连接、jar包，排查文件句柄泄露（too many open files）。

10. Linux查看Java进程内存占用

```
# top 实时看，VIRT虚拟内存，RES物理内存
top -p <pid>
# ps看内存占比 %MEM
ps -aux | grep <pid>
# jstat -gc <pid> 看堆内GC内存
```

11. 输入URL到页面返回全过程（标准版本）

1. DNS解析：域名 → IP
2. 建立TCP三次握手连接（HTTPS还要TLS握手）
3. 浏览器发送HTTP请求
4. 服务器Nginx/应用层接收请求，路由、业务处理
5. 服务器返回HTTP响应报文
6. 浏览器收到响应，关闭TCP连接（四次挥手）
7. 浏览器解析HTML，渲染页面、加载CSS/JS、图片等静态资源

12. DNS全套问题汇总

1. DNS端口：**UDP 53**，大包会走TCP 53
2. 递归查询流程：浏览器缓存 → 本地hosts文件 → 本地DNS服务器 本地DNS找不到，向**根域名服务器**发起迭代查询：根服务器返回顶级域服务器地址 → 顶级域返回二级域服务器 → 直到拿到目标IP
3. 常见DNS： 114.114.114.114（国内）、8.8.8.8（谷歌）、223.5.5.5（阿里）
4. 本地hosts配置位置： Linux：`/etc/hosts` Windows：`C:\Windows\System32\drivers\etc\hosts`

> 区分：**递归查询（本地DNS替客户端查）、迭代查询（本地DNS一层层问根、顶级域）**

13. 手撕：二分查找（Java，可直接写在白板）

```
public class BinarySearch {
    public static int binarySearch(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2; // 防止left+right溢出
            if (arr[mid] == target) {
                return mid;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return -1;
    }

```

```
    public static void main(String[] args) {
        int[] nums = {1,3,5,7,9,11};
        System.out.println(binarySearch(nums,7));
    }
}
```

要点：`left <= right`；mid防溢出写法；**二分只适用于有序数组。**

微步在线3一面

> 适配后端面试，兼顾原理+考点，可直接口述

1、项目问了点

> 提示：面试官会深挖：项目背景、你的职责、遇到什么难点、怎么排查、优化手段、结果收益。
> 通用回答框架（你套自己的实习/项目）：
> 1）项目是做什么，业务背景；
> 2）我承担模块，技术栈；
> 3）遇到的技术难点（比如慢查询、并发问题、缓存一致性）；
> 4）自己的解决方案；
> 5）最终效果、数据（QPS提升、耗时下降）；
> 6）反思：还有哪些可以改进。

2、线程池的参数

`ThreadPoolExecutor`七大参数：

1. **corePoolSize**：核心线程数，一直保留，不会回收
2. **maximumPoolSize**：最大线程数，核心满了，任务队列满之后才创建非核心线程
3. **keepAliveTime**：非核心线程空闲存活时间
4. **unit**：时间单位
5. **workQueue**：阻塞队列，存等待执行的任务（ArrayBlockingQueue、LinkedBlockingQueue等）
6. **threadFactory**：线程工厂，创建线程，可以设置线程名
7. **handler**：拒绝策略，队列+最大线程都满了执行

- AbortPolicy：抛异常（默认）
- DiscardPolicy：直接丢弃任务
- DiscardOldestPolicy：丢弃队列队首任务，执行当前
- CallerRunsPolicy：调用者线程执行任务

执行流程：
来了任务 → 核心线程数未满，新建核心线程；核心满→放入阻塞队列；队列满→新建非核心线程直到`maximumPoolSize`；达到最大线程，执行拒绝策略。

3、MySQL InnoDB 和 MyISAM区别

对比	InnoDB	MyISAM
事务	支持ACID事务	不支持事务
锁	行锁+表锁	只有表锁
外键	支持	不支持
崩溃恢复	有redo/undo日志，崩溃安全	无，宕机可能损坏数据
索引	聚簇索引，主键索引存整行数据	非聚簇，索引和数据分开
MVCC	支持，实现隔离级别	不支持
count(*)	需要扫描表	保存行数，count很快

> 生产环境基本全部使用InnoDB；MyISAM适合只读静态表，现在很少用。

4、TCP和UDP的区别，TCP如何保证可靠

区别

1. TCP面向连接；UDP无连接
2. TCP可靠传输；UDP不可靠
3. TCP慢，有握手、重传、拥塞控制；UDP开销小速度快
4. TCP流式；UDP面向数据报
5. TCP有拥塞控制；UDP没有

TCP可靠性保证手段

1. 序列号+ACK确认应答
2. 超时重传，丢包重发
3. 滑动窗口，流量控制
4. 拥塞控制，防止网络过载
5. 校验和，校验数据损坏

5、UDP对应的应用层协议

- DNS（大部分UDP53）
- DHCP
- TFTP简单文件传输
- QUIC（HTTP3底层）
- RTP/RTSP音视频直播

6、如何保证三个线程的执行顺序（A→B→C）

方案1：`join()`，A线程执行完，调用A.join()，B再执行；B执行完B.join()，C执行。简单直观。
方案2：CountDownLatch闭锁，A完成计数器减一，B等待；B完成再减一，C等待。适合多线程。
方案3：单线程池，只有一个线程，任务按顺序提交，天然串行。
方案4：Semaphore信号量，设置许可控制放行。

> 不推荐sleep，不稳定，靠时间不可靠。

###7、判断链表有环（快慢指针）
思路：快慢指针，slow每次走一步，fast每次走两步；
如果链表无环，fast会走到null；有环，快慢指针一定会在环内相遇。
扩展：相遇后，一个指针回到头节点，两个指针同速前进，相遇点就是环入口。

```java
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while(fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if(slow == fast) return true;
    }
    return false;
}
```

###8、重写和复写的区别

> 术语：**重写(Override，覆写)、重载(Overload)**，面试官口误叫复写。

- **重写Override**：发生父子类；方法名、参数列表、返回值一致；子类重写父类非static方法；运行期动态绑定。
- **重载Overload**：同一个类内部；方法名相同，参数列表不同；返回值不做区分；编译期静态绑定。

###9、try-catch-finally执行顺序

1. try块执行；发生异常跳到catch；无异常跳过catch。
2. **无论是否异常，return，finally一定会执行。**>

> 坑：如果JVM退出`System.exit(0)`，finally不会执行。
> 坑点：try里面return，会先把返回值保存，执行完finally，再return；finally里修改返回基本类型变量，不会改变返回值；修改对象属性会生效。

###10、Linux的一些命令（高频）
进程：`ps -ef`、`top`、`pidstat`
文件：`ls`、`cat`、`grep`、`less`、`tail -f`
网络：`ss`、`netstat`、`lsof`
权限：`chmod chown`
查找：`find`
压缩：`tar`
重定向：`>`覆盖，`>>`追加

###11、快排的思想
分治思想：

1. 在数组选一个基准pivot；
2. partition分区：小于pivot放左边，大于放右边；
3. 递归对左子数组、右子数组重复分区排序。
   时间复杂度：平均O(nlogn)；最坏O(n^2)（已经有序）；不稳定排序。

###12、红黑树和AVL树
两者都是自平衡二叉搜索树。

- **AVL树**：左右子树高度差不超过1；平衡严格；旋转次数多；查询快，插入删除开销大。
- **红黑树**：不追求严格平衡，通过红黑五条规则维持大致平衡；最大高度2logn；插入删除旋转少，性能更好；HashMap、TreeMap底层使用。

> 对比总结：查询多选AVL；插入删除频繁优先红黑树。

微步在线3二面

###1、聊项目，项目做的目的，技术难点，怎么解决的
口述模板：
项目目的：业务上要解决什么痛点。
难点：例如大数据量、并发高、数据库慢、缓存一致性。
我的动作：做了什么分析，采取的方案（索引、缓存、异步、分库分表等）。
结果：耗时下降多少，QPS提升，解决了什么问题。
反思：还存在哪些不足。

###2、重写和重载（同上一面第8题）

###3、HashMap put过程（JDK8）

1. 计算key的hash值（扰动函数高低位混合）；
2. 判断table数组是否初始化，没有则扩容初始化；
3. 通过`hash & (n-1)`定位桶下标；
4. 如果桶为空，直接新建Node放入；
5. 如果桶不为空：
    - 桶首节点key相等（hash相同&&equals），覆盖旧value；
    - 如果是红黑树节点，走红黑树插入；
    - 如果是链表，遍历链表末尾插入；链表长度>=8，数组长度>=64转为红黑树；
6. put完成，判断size是否大于阈值threshold，超过则执行resize扩容。

###4、HashMap的实现（JDK8）
底层：数组 + 链表 + 红黑树。

- 数组table，存放桶；
- 链表解决hash冲突；链表过长转红黑树提高查询效率；节点数小于等于6退化成链表。
- 容量必须是2的幂，`hash & (n-1)`代替取模；
- 负载因子默认0.75，阈值=容量*负载因子，达到阈值触发扩容，扩容为原来2倍。

###5、CAS
Compare And Swap，比较并交换，乐观锁。
三个参数：内存地址V，预期旧值A，新值B。
逻辑：如果内存V的值等于预期A，就更新为B；否则更新失败。
Java：Unsafe底层CPU指令实现；AtomicXXX系列底层CAS。
问题：

1. ABA问题（值A→B→A，CAS误认为没变化，版本号解决）
2. 自旋空转，CPU消耗大
3. 只能保证一个变量原子性

###6、`0.1*3==0.3`和`0.3*1 ==0.3`哪个true/false

> 二进制浮点数精度丢失
> `0.1*3 == 0.3` → **false**
> `0.3*1 ==0.3` → **true**
> 解释：0.1二进制无法精确表示，0.1*3计算后得到近似值不等于0.3；0.3*1本身就是同一个浮点数。
> 业务处理小数金额，不用double/float，使用BigDecimal。

###7、Integer和int

1. int是基本数据类型；Integer是包装类对象。
2. Integer有缓存，-128~127，这个范围`==`比较返回true；超出缓存范围会新建对象，`==`比较地址不等。

```
Integer a=100; Integer b=100; //true
Integer c=200; Integer d=200; //false
```

3. 自动装箱拆箱：int自动包装Integer；Integer拆箱为int。>

> 比较Integer对象尽量用equals，不要直接==。

###8、面向对象的理解
四大特性：封装、继承、多态、抽象。

1. **封装**：把数据和行为封装类内部，对外暴露接口，隐藏内部实现。
2. **继承**：子类复用父类代码，扩展功能，is-a关系。
3. **多态**：同一行为不同实现；重写实现运行时多态。
4. **抽象**：抽象类、接口，定义规范，不关心具体实现。
   好处：易维护、易扩展、代码复用。

###9、字符串最长无重复子串（滑动窗口）

> LeetCode3，手撕思路
> 滑动窗口left、right双指针；用Set或者HashMap记录字符最近出现下标。right不断向右；遇到重复字符移动left，保证窗口没有重复字符，记录窗口最大长度。
> 时间复杂度O(n)。

```
public int lengthOfLongestSubstring(String s) {
    HashMap<Character,Integer> map = new HashMap<>();
    int maxLen = 0;
    int left = 0;
    for(int right =0;right<s.length();right++){
        char c = s.charAt(right);
        if(map.containsKey(c)){
            left = Math.max(left, map.get(c)+1);
        }
        map.put(c,right);
        maxLen = Math.max(maxLen, right-left+1);
    }
    return maxLen;
}
```

###10、海量数据的排序（内存放不下全部数据，外部排序）
外部排序：

1. 分块：把大文件切分成多个小文件，每块能加载进内存；内存内部快排，写回磁盘。
2. 多路归并：把多个有序小文件进行k路归并，输出最终有序大文件。
   优化：败者树减少归并比较开销。

###11、shell命令 `ls >a`
`ls` 列出当前目录文件；
`>` 输出重定向，**覆盖**，把ls的标准输出写入文件 `a`；文件a不存在就创建；存在直接清空原有内容写入。
补充：`>>` 是追加，不会覆盖原有内容。
区分：`2>` 重定向错误输出；`&>` 标准输出+错误输出。

微步4

1. Object有哪些方法

```
// 1. native本地方法
native void registerNatives()
native Class<?> getClass()
native int hashCode()
native void wait(long timeout)
native void notify()
native void notifyAll()
// 2. 普通方法
boolean equals(Object obj)
String toString()
void wait()
void wait(long timeout, int nanos)
// 3. finalize（JDK9废弃）
protected void finalize()
// 4. clone
protected Object clone() throws CloneNotSupportedException
```

口述精简：
`getClass()`、`hashCode()`、`equals()`、`toString()`、`clone()`、`finalize()`、`wait()`、`notify()`、`notifyAll()`

2. equals和hashCode为什么要一起重写

> 核心约定：**equals返回true的两个对象，hashCode必须相等；hashCode相等，equals不一定true**

1. 如果只重写equals、不改hashCode：两个逻辑相等的对象，hashCode大概率不一样，HashMap/HashSet会分到不同桶，集合认为是两个不同对象，出现重复存储、查不到的bug；
2. 如果只重写hashCode、不改equals：hash碰撞时，equals返回false，集合能正常区分，但违背规范，可读性差，容易出隐患。

3. 介绍HashMap（JDK8）

底层：**数组 + 链表 + 红黑树**

1. 容量必须是2的n次幂，用 `hash & (n-1)` 替代取模，提升效率；
2. hash扰动函数：`h = key.hashCode() ^ (h >>> 16)`，让高位参与运算，减少碰撞；
3. 负载因子默认0.75，`阈值threshold = 容量 * 0.75`，超过阈值触发2倍扩容；
4. put定位桶：
    - 桶为空：直接放节点
    - 桶首key相等：覆盖value
    - 桶是红黑树：走树插入
    - 桶是链表：尾部追加；链表长度≥8 && 数组长度≥64 → 转红黑树；节点≤6退化成链表；
5. 线程不安全：并发put会丢数据、链表环，并发场景用ConcurrentHashMap。

4. 常用JDK版本，Java8/11/17新特性

我平时开发、项目里主要用 **JDK8**，也了解JDK11、JDK17（LTS长期支持版）

- **JDK8（最重要）**：Lambda、Stream流、函数式接口、Optional、新时间API（LocalDateTime）、HashMap红黑树、CompletableFuture、Base64、PermGen永久代移除，换成元空间Metaspace（堆外内存）

- **JDK11（LTS）**：HTTP Client API、var局部变量类型推断、ZGC初始版本、废弃Applet、移除Java EE包、单文件源码直接`java xxx.java`运行
- **JDK17（LTS，最新长期支持）**：密封类Sealed Classes、模式匹配Pattern Matching for instanceof、虚拟线程Virtual Thread（JDK17孵化，JDK21正式）、强封装，默认禁止反射访问内部API、ZGC稳定、Vector API

> 加分补充：企业选型：8存量最多，新项目很多开始上17；11是过渡LTS。

5. Java异常分为哪几种

Throwable 是顶层父类，分两大分支：

1. **Error（错误）**：系统级严重错误，程序无法处理：OOM、StackOverflowError、虚拟机崩溃；
2. **Exception（异常）**：程序层面可捕获处理，又分：
    - 受检异常（Checked Exception）
    - 非受检异常（Unchecked Exception，RuntimeException及其子类）

6. 受检异常和非受检异常使用区别

- **受检异常**：编译期强制检查，要么`try-catch`捕获，要么方法上`throws`抛出给上层调用方；比如IOException、SQLException；用于外部可预期失败（文件不存在、网络IO失败）
- **非受检异常（运行时异常）**：编译器不强制捕获/声明；比如NullPointerException、IndexOutOfBoundsException；大多是代码bug导致。>

> 开发规范：业务上不要随意抛RuntimeException，框架底层常用；受检异常不要滥用，避免大量throws污染方法签名。

7. Java对象的创建过程（new）

1. 类加载检查：先检查类是否加载、链接、初始化，没有就先执行类加载；
2. 分配内存：在堆划分一块内存（TLAB优先分配）；
3. 初始化零值：内存全部赋默认零值（int=0，引用=null）；
4. 设置对象头：markWord、klass指针、哈希、锁标记等；
5. 执行构造方法：初始化成员变量、执行构造代码块、执行构造函数；
6. 把引用赋值给栈上变量。

8. 对象内存分配在哪？什么情况堆溢出

- 分配位置：**优先堆Heap**；开启逃逸分析，小对象、不会逃逸的对象，JIT会做**栈上分配**；
- 堆OOM常见场景：
    1. 集合/静态List/Map持续add元素，没有清理，内存泄漏；
    2. 大对象，一次性加载超大文件、大批量数据到内存；
    3. 线程无限创建对象，循环new不释放；
    4. 内存泄漏：ThreadLocal不remove、长生命周期对象持有短生命周期对象引用。

9. GC为什么回收不掉OOM对象？

GC回收的前提：**对象没有有效可达引用链。**
OOM的时候，这些对象依然被GC Roots可达（静态变量、栈引用、ThreadLocal等持有），属于存活对象，GC不能回收；不是GC坏了，是业务代码一直持有引用，对象无法被判定为垃圾，堆空间被存活对象占满，触发OOM。

10. 除了强引用，还有哪几种引用

一共4种：

1. **强引用**：`Object o = new Object()`；只要引用存在，GC永远不回收；
2. **软引用 SoftReference**：内存不够的时候GC才回收，适合缓存；
3. **弱引用 WeakReference**：只要发生GC，一定会被回收；WeakHashMap；
4. **虚引用 PhantomReference**：最弱，无法通过引用拿到对象，只用来收到回收通知，配合Cleaner做资源释放。

11. CPU 100%排查流程（Java线上）

1. `top` 找到占用CPU高的Java进程PID；
2. `top -H -p pid` 看这个进程里面哪个线程TID占用CPU高；
3. printf把TID转成16进制；
4. `jstack pid` 打印线程栈，匹配十六进制TID，定位代码；
5. 分析：死循环、大量自旋CAS、频繁GC（CPU高但是业务慢，看GC）；
6. 辅助：`jstat -gc`看GC情况、Arthas `thread`命令一键定位。

> 两种典型：①业务死循环（us高）；②疯狂FullGC（gc线程占满CPU）

12. synchronized修饰普通方法、静态方法区别

本质是锁对象不一样：

- 普通成员方法：锁是**当前实例对象 this**；同一个对象调用同步方法互斥；不同实例互不影响；
- static静态方法：锁是**类的Class对象**；所有实例共用一把类锁；>

> 类锁和对象锁互不阻塞。

13. synchronized代码块里可以锁String或者Integer吗?

语法上允许，但是工程里强烈不推荐!

1. String：字符串常量池，`"abc"`字面量复用，不同地方锁同一个字符串，意外锁竞争、死锁；
2. Integer：-128~127缓存，`Integer i=10`多处使用会拿到同一个对象；
   ✅ 最佳实践：new一个专门的 `private final Object lock = new Object();` 作为锁对象。

14. CopyOnWriteArrayList怎么保证线程安全

写时复制 COW
读不加锁；写（add/set/remove）的时候：先拿到锁，复制原有数组，在新数组上修改，修改完成后，volatile引用指向新数组。
优点：读极快，适合读多写少；
缺点：写的时候拷贝数组，内存开销大，大批量写性能差；有数据弱一致性（读的是旧数组快照）。

15. 有没有CopyOnWriteLinkedList？为什么没有

JDK没有提供。
COW核心代价是**全数组拷贝**：ArrayList是连续数组，拷贝成本可控；
LinkedList是链表，如果做写时复制，修改一个节点就要拷贝整条链表，拷贝开销巨大，性能完全不可接受，所以没有这个实现。

16. 线程池7大参数（ThreadPoolExecutor）

`corePoolSize`核心线程、`maximumPoolSize`最大线程、`keepAliveTime`非核心空闲时间、时间单位unit、阻塞队列workQueue、线程工厂threadFactory、拒绝策略handler。
执行流程：任务来了→核心线程未满新建核心；核心满→入队列；队列满→创建非核心直到max；达到max执行拒绝策略。

17. core=8，max=10，能不能不进队列，先跑满10个线程再入队？

原生ThreadPoolExecutor默认策略不能，原生是先队列、再扩容线程
原生逻辑：核心满了优先丢队列，队列满才开非核心线程。
想要改成：**先开线程直到max，再入队列**，有两种办法：

1. 自定义阻塞队列，重写offer方法，返回false，让线程池认为队列满，去创建非核心线程；
2. 用第三方工具类，或者自己改造线程池逻辑。>

> 注意：这种模式风险高，流量突刺会瞬间创建大量线程，容易打垮服务，一般不推荐。

如果你想要，我可以把这一批全部压缩成【一句话背诵精简版】。

微步5

1. 自我介绍+实习介绍

> 给你适配版本，直接口述，结合你简历：
> 面试官您好，我是XX，XX学校电子信息方向研究生，求职Java后端开发。
> 实习期间在亚信科技参与香港HRMS人力中台项目，主要使用SpringBoot + SpringCloud Alibaba、MySQL、Redis做后端开发，负责考勤、薪资试算、分页查询接口开发，处理过慢SQL优化、并发和数据一致性问题。
> 同时个人项目里基于SpringAI做了RAG知识库问答系统，接入向量库、Embedding模型，落地了MCP协议，实现大模型调用业务工具，做知识库检索和业务数据混合查询，熟悉向量检索、重排、父子切片这些RAG链路。
> 平时重点复习JUC、JVM、MySQL索引、分布式、网络操作系统基础，习惯用Arthas、Jstack、Linux命令做线上问题排查，对高并发、高性能后端开发感兴趣。

2. Spring AI架构，组件和功能

SpringAI是Spring生态面向大模型应用开发的框架，核心目标是统一屏蔽不同大模型厂商API差异，标准化RAG、工具调用、提示词能力。
主要组件：

1. **Model（模型抽象）**：ChatModel、EmbeddingModel、ImageModel，统一接口，切换OpenAI/通义/本地模型不用改业务代码；
2. **Prompt 提示词**：PromptTemplate，模板化管理提示词，支持变量填充；
3. **Output Parser 输出解析器**：把大模型返回文本解析为POJO、JSON，解决输出格式不可控；
4. **VectorStore 向量存储**：向量库抽象接口，对接Milvus、Chroma、RedisVector等，存文档向量，做相似度检索；
5. **Document Reader 文档读取器**：读取PDF、Markdown、文本文件，加载原始文档；
6. **Document Transformer 文档切片/转换器**：文本分割、父子切片、清洗；
7. **Retriever 检索器**：向量检索、BM25检索、混合检索、Reranker重排；
8. **Tool / Function Calling 工具调用**：让大模型自主选择调用后端Java业务接口；
9. **ChatMemory 会话记忆**：保存对话历史，实现多轮对话；
10. **Advisor 拦截器**：在请求大模型前后做切面，日志、校验、权限、RAG注入上下文。

一句话总结：从文档加载、切片、向量化存储、检索、提示词组装、模型调用、工具调用、结果解析，一整条RAG应用链路封装。

3. 项目里MCP是什么，详细说明

MCP = Model Context Protocol，**模型上下文协议**，是标准化协议，用来让大模型安全调用外部工具、获取外部上下文。

> 区分：不是SpringAI自带，是独立标准，解决Function Calling各家格式不统一的痛点。
> 架构分三端：

- **MCP Host（宿主）**：大模型应用，也就是我们的SpringAI服务；
- **MCP Client（客户端）**：嵌入在我们应用内，和MCP Server通信；
- **MCP Server（服务端）**：提供能力的服务，暴露工具、资源、提示词模板。

核心能力：

1. **Tools工具**：可调用函数（比如查票务、查规章、查数据库接口），服务端声明入参、描述，大模型自主判断什么时候调用；
2. **Resources资源**：结构化数据源、文档、知识库，模型可以读取；
3. **Prompts提示词模板**：服务端托管可复用prompt；

通信方式：stdio标准输入输出、HTTP SSE。
和原生Function Calling对比：
原生Function Calling是模型厂商自定义格式，换模型就要改代码；MCP是通用标准，一套工具可以被任意支持MCP的大模型调用，解耦大模型和业务工具。
在我的剧院票务项目：把选座查询、规章知识库封装成MCP Server提供工具，SpringAI作为Host接入，大模型回答用户问题时，按需调用业务接口+向量知识库，混合回答。

4. 提效方面团队实践探索（实习+项目两个角度）

业务开发提效

1. 统一基础脚手架：封装公共返回、异常处理、权限、分页、Redis工具，减少重复样板代码；
2. 统一SQL规范、慢SQL平台，上线自动审计，提前发现索引问题；
3. 封装通用组件：字典转换、文件导出、消息队列生产者消费者模板；
4. CI/CD流水线：Jenkins+Docker自动构建、部署、回滚，自动化测试脚本；

AI提效探索（贴合SpringAI项目）

1. 用RAG做内部文档问答，查接口文档、数据库字典、开发规范；
2. MCP封装常用运维/查询工具，自然语言查日志、查数据库；
3. 代码辅助：大模型生成单元测试、写DTO、写接口文档、解释报错堆栈；
4. 提示词沉淀，把常见排查脚本、正则、SQL模板托管，复用。

5. 美团现在使用的是哪个出码工具?

> 这题属于业务常识，校招一般不深挖，标准答案：
> 美团自研的出码工具：**MDD（美团低代码平台），内部代码生成平台，也有自研脚手架、基于AST的代码生成工具。**
> 补充：业界通用出码：MyBatis Generator、CodeGen，美团内部自研，不对外开源。
> 如果你不知道，稳妥回答：我了解美团内部有自研低代码和代码生成平台MDD，用于前后端代码生成；外部开源工具我用过MyBatis Generator、SpringBoot代码生成器。

6. 大模型如何帮助后端开发提效、改变传统开发

1. **编码辅助**：生成CRUD、DTO、单元测试、异常处理、JUC多线程Demo，补齐样板代码；
2. **问题排查**：粘贴报错堆栈、Arthas日志，辅助定位空指针、死锁、GC异常、SQL问题；
3. **文档**：接口注释、README、需求文档、数据库字典自动生成；
4. **架构和方案**：帮梳理限流、缓存、分布式锁方案，对比技术选型；
5. **RAG+MCP**：把内部接口、数据库、运维命令封装成工具，自然语言查询业务数据、查知识库；
6. **测试辅助**：构造测试用例、边界case，造压测脚本；

但是不会替代后端：大模型不懂业务边界、分布式一致性、线上故障风险，最终校验、架构设计、线上稳定性依然由开发把控。
传统方式：查文档、复制样板代码、手动写单元测试；现在可以把重复性工作交给模型，人聚焦难点：并发、数据一致性、容量规划、故障预案。

7. 0~1000万乱序整数，内存只能放下250万，排序输出文件（外部排序）

这是经典**外部排序**，步骤口述：
总数据：1000万，内存容纳250万。

1. **分块（拆分）**：分批读取文件，每次读250万整数，在内存内快速排序，写入临时有序小文件。一共需要拆成 4 个有序子文件；
2. **多路归并**：开启4路归并，每个文件维护一个指针，每次取四路当前最小数字，写入最终结果文件；
3. 优化点：
   - 整数可以用位图BitMap优化（本题是0~1000万，范围已知！可以优先提位图方案，加分）
     ✅ 位图方案（最优）：
     0~1000万，一共10000001个数，一个bit标记是否存在；
     1000万bit ≈ 1.2MB，内存完全足够！遍历所有数字标记bit，最后顺序遍历bit，存在就输出。

> 口述优先先说位图：因为给定数值范围0~1000万，非常适合BitMap，远优于外部归并；如果没有数值范围，才用外部多路归并。

> 面试官追问：如果数字范围很大，不能用位图怎么办？再讲多路归并外部排序。

8. 是否熟悉Linux主要命令

分几类，挑高频的说，结合Java排查：
进程：`ps -ef`、`top`、`pidstat`、`jps`
文件查看：`cat`、`less`、`tail -f`、`grep`、`find`
网络：`ss -tlnp`、`lsof`、`netstat`
权限：`chmod`、`chown`
重定向管道：`|`、`>`、`>>`
压缩：`tar`
编辑：`vim`
我线上排查Java问题经常组合：`top`看CPU，`lsof`看文件句柄，`ss`看连接，配合jstack、jstat。

9. 数据库用过哪些

关系型：MySQL（InnoDB为主，项目主力），了解PostgreSQL；
非关系型：Redis（缓存、分布式锁、限流），MongoDB；向量库Milvus（SpringAI RAG项目）。

10. 关系型（RDB）和非关系型（NoSQL）区别

1. **数据模型**
   关系型：二维表，行+列，Schema固定；
   NoSQL：键值、文档、列簇、图，Schema灵活；
2. **事务**
   MySQL等RDB：完整ACID，强事务；
   大部分NoSQL默认弱事务，支持最终一致性；
3. **查询语言**
   RDB：SQL；
   NoSQL：自有API；
4. **适用场景**
   关系型：金融、人力、订单，数据关联多，需要事务；
   NoSQL：缓存、海量日志、文档、向量检索、高吞吐简单读写；
5. **扩展性**
   NoSQL天然水平分片容易；MySQL分库分表需要额外成本。

如果你想要，我把这一组，加上前面所有微步题目合并成一份总复习文档，或者给你做一份45秒精简背诵版。